$\sum$

A Library for ANSI Common Lisp

https://github.com/cgore/sigma/

Christopher Mark Gore
http://www.cgore.com
cgore@cgore.com

Sunday, August 9th, AD 2026

Contents

Copyright

Copyright © 2005 – 2026, Christopher Mark Gore,
Soli Deo Gloria,
All rights reserved.

22 Forest Glade Court, Saint Charles, Missouri 63304 USA.
Web: `http://www.cgore.com`
Email: cgore@cgore.com

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Christopher Mark Gore nor the names of other contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Introduction

The Σ library is a generic library of mostly random useful code for ANSI Common Lisp. It is currently only really focused on SBCL, but patches to add support for other systems are more than welcome.

This library started out as a single file, `utilities.lisp`, that I personally used for shared generic code for all of my Lisp code. Most lispers have a similar file of some name, `utilities.lisp`, `misc.lisp`, `shared.lisp`, or even `stuff.lisp`, that is just a random collection of useful little generic macros and functions. Mine has grown over the years, and in 2012 I decided that I should try to make it useful to people other than myself.

You can download the library from GitHub at:
`https://github.com/cgore/sigma`
and I have some other information on it at my own website at:
`http://cgore.com/programming/lisp/sigma/`

Getting Lisp

Before using this library you need a working Lisp. I use and recommend SBCL, Steel Bank Common Lisp, which is available at:
`http://www.sbcl.org`
This is derived from CMUCL, Carnegie Mellon University Common Lisp, which is still under active development and is available at:
`http://www.cons.org/cmucl/`

SBCL has information on getting started at:
`http://www.sbcl.org/getting.html`
If you are using Debian or a similar Linux distribution (including Ubuntu), you can just run as root:

```
apt-get install sbcl sbcl-doc sbcl-source
```

Getting EMACS and SLIME

After installing, the best way to interact with any Common Lisp is via SLIME, the Superior Lisp Interaction Mode for EMACS, which is avail-

able at:
`http://common-lisp.net/project/slime/`
This can be installed on Debian by:
`apt-get install slime emacs emacs-goodies-el`

Using the Library

First we need to clone the utilities.

```
mkdir -p  /programming/lisp
cd  /programming/lisp
git clone git@github.com:cgore/sigma.git
```

Now we need to make a directory for our project and symlink to the ASDF definition. There are other ways to load ASDF libraries, especially if you want to have them available globally; I strongly recommend you read the documentation to ASDF.

```
mkdir our-new-project
cd our-new-project
ln -s  /programming/lisp/sigma/sigma.asd
```

Now we need to start up our Lisp REPL. The best way to do this for personal use is SLIME from within Emacs, but I will demonstrate using the shell itself here.

```
sbcl
```

Now we are in SBCL. Let's calculate something.

$$\sum_{i=0}^{100} i$$

```
(asdf:load-system :sigma) ; Load the Sigma system via ASDF.
(sigma:use-all-sigma) ; This will pollute COMMON-LISP-USER.
(sum (loop for i from 1 to 100
           collect i)) ; Returns 5050, and makes Euler sad.
```

Have fun!

Chapter 1

The `sigma/behave` Package

The `sigma/behave` package contains some useful code for confirming behavior of code, supporting a very basic form of *behavior-driven development*, BDD. The basic flow is to define the *behavior* of something, with multiple *specs* specified within that behavior specification, each consisting of various assertions, such as `should=`, `should-equal`, `should-not-equal`, and many others. If the behavior of the thing doesn't match the specified behavior, then there is some error.

1.1 Macros

1.1.1 The `behavior` Macro

The `behavior` macro is used to specify a block of expected behavior for a `thing`. It specifies an example group, loosely similar to the `describe` blocks in Ruby's RSpec. It takes a single argument, the `thing` we are trying to describe, and then a body of code to evaluate that is evaluated in an implicit `progn`. It is to be used around a set of examples, or around a set of assertions directly.

Syntax

`(behavior` *`thing`* `&body` *`body`*`)`

Arguments and Values

`thing` This is what we are describing the behavior of.

`body` This is an implicit `progn` to contain the behavior.

Examples

```
(behavior 'float
          (spec "is an Abelian group"
                (let ((a (random 10.0))
                      (b (random 10.0))
                      (c (random 10.0))
                      (e 1.0))
                  (spec "closure"
                        (should-be-a 'float (* a b)))
                  (spec "associativity"
                        (should= (* (* a b) c)
                                 (* a (* b c))))
                  (spec "identity element"
                        (should= a (* e a)))
                  (spec "inverse element"
                        (let ((1/a (/ 1 a)))
                          (should= (* 1/a a)
                                   (* a 1/a)
                                   1.0)))
                  (spec "commutativity"
                        (should= (* a b) (* b a))))))
```

1.1.2 The spec Macro

The spec macro is used to indicate a specification for a desired behavior. It will normally serve as a grouping for assertions or nested specs.

Syntax

(spec *description* &body *body*)

Arguments and Values

description This is a string to describe the specification.

body This is an implicit progn to contain the specification.

Examples

```
(spec "should pass some tests"
      (should= 12 (foo 3.5))
      (should= 14 (foo 4.22)))
```

1.1.3 The `should` Macro

The `should` macro is the basic building block for most of the behavior checking. It asserts that `test` returns truthfully for the arguments. Typically you will want to use one of the macros defined on top of `should` instead of using it directly, such as `should=`.

Syntax

`(should` *`test`* `&rest` *`arguments`*`)`

Arguments and Values

`test` This is the test predicate to evaluate.

`arguments` These are the arguments to the test predicate.

Examples

```
(should #'= 12 (* 3 4)) ; Passes
(should #'< 4 (* 2 3))  ; Passes
(should #'< 4 5 6 7)    ; Passes
```

1.1.4 The `should-not` Macro

The `should-not` macro is identical to the `should` macro, except that it inverts the result of the call with `not`.

Syntax

`(should-not` *`test`* `&rest` *`arguments`*`)`

Arguments and Values

`test` This is the test predicate to evaluate.

`arguments` These are the arguments to the test predicate.

Examples

```
(should-not #'< 12 4)  ; Passes
(should-not #'= 12 44) ; Passes
```

1.1.5 The `should-be-null` Macro

The `should-be-null` macro is a short-hand method for `(should #'null ...)`.

Syntax

`(should-be-null &rest arguments)`

Arguments and Values

arguments These are the arguments to `null`.

Examples

```
(should-be-null ())            ; Passes
(should-be-null nil)           ; Passes
(should-be-null (not 12))      ; Passes
(should-be-null (and t t nil)) ; Passes
```

1.1.6 The `should-be-true` Macro

The `should-be-true` macro is a short-hand method for `(should #'identity ...)`.

Syntax

`(should-be-true &rest arguments)`

Arguments and Values

arguments These are the arguments to `identity`.

Examples

```
(should-be-true t)               ; Passes
(should-be-true (not nil))       ; Passes
(should-be-true (or nil nil 12)) ; Passes
```

1.1.7 The `should-be-false` Macro

The `should-be-false` macro is a short-hand method for `(should #'not ...)`.

Syntax

`(should-be-false &rest arguments)`

Arguments and Values

arguments These are the arguments to `not`.

Examples

```
(should-be-false nil)
(should-be-false (not t))
(should-be-false (< 44 2))
```

1.1.8 The `should-not-be-true` Macro

The `should-not-be-true` macro is a short-hand method for `(should-not #'identity ...)`. It asserts that the body does not evaluate to a true value.

Syntax

`(should-not-be-true &body` *body*`)`

Examples

```
(should-not-be-true nil)      ; Passes
(should-not-be-true (not t)) ; Passes
(should-not-be-true t)       ; Fails
```

1.1.9 The `should-not-be-false` Macro

The `should-not-be-false` macro is a short-hand method for `(should-not #'not ...)`. It asserts that the body does not evaluate to a false value; that is, that it is true.

Syntax

`(should-not-be-false &body` *body*`)`

Examples

```
(should-not-be-false t)         ; Passes
(should-not-be-false (not nil)) ; Passes
(should-not-be-false nil)       ; Fails
```

1.1.10 The `should-be-a` Macro

The `should-be-a` macro specifies that one or more `things` should be of the type specified by `type`.

Syntax

(should-be-a *type* &rest *things*)

Arguments and Values

type This is the type to compare with via typep.

things These are the things to confirm the type of.

```
(should-be-a 'integer 1)                 ; Passes
(should-be-a 'float 1.0)                 ; Passes
(should-be-a 'integer 1 2 3 4 5 6 7 8 9) ; Passes
(should-be-a 'integer 1.0)               ; Fails
```

1.1.11 The should= Macro

The should= macro is a short-hand method for (should #'= ...).

Syntax

(should= &rest *arguments*)

Arguments and Values

arguments These are the arguments to =.

Examples

```
(should= 12 12)   ; Passes
(should= 12 12.0) ; Passes
```

1.1.12 The should-not= Macro

The should-not= macro is a short-hand method for (should-not #'= ...).

Syntax

(should-not= &rest *arguments*)

Arguments and Values

arguments These are the arguments to =.

Examples

```
(should-not= 12 12)   ; Fails
(should-not= 12 12.0) ; Fails
(should-not= 12 14)   ; Passes
```

1.1.13 The `should/=` Macro

The `should/=` macro is a short-hand method for `(should #'/= ...)`.

Syntax

`(should/= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `/=`.

Examples

```
(should/= 12 13)   ; Passes
(should/= 12 12)   ; Fails
(should/= 12 12.0) ; Fails
```

1.1.14 The `should-not/=` Macro

The `should-not/=` macro is a short-hand method for `(should-not #'/= ...)`.

Syntax

`(should-not/= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `/=`.

Examples

```
(should-not/= 12 13)   ; Fails
(should-not/= 12 12)   ; Passes
(should-not/= 12 12.0) ; Passes
```

1.1.15 The should< Macro

The should< macro is a short-hand method for (should #'< ...).

Syntax

(should< &rest *arguments*)

Arguments and Values

arguments These are the arguments to <.

Examples

```
(should< 12 13) ; Passes
(should< 13 12) ; Fails
(should< 12 12) ; Fails
```

1.1.16 The should-not< Macro

The should-not< macro is a short-hand method for (should-not #'< ...).

Syntax

(should-not< &rest *arguments*)

Arguments and Values

arguments These are the arguments to <.

Examples

```
(should-not< 12 13) ; Passes
(should-not< 13 12) ; Fails
(should-not< 12 12) ; Fails
```

1.1.17 The should> Macro

The should> macro is a short-hand method for (should #'> ...).

Syntax

(should> &rest *arguments*)

Arguments and Values

arguments These are the arguments to >.

Examples

```
(should> 12 13) ; Fails
(should> 13 12) ; Passes
(should> 12 12) ; Fails
```

1.1.18 The `should-not>` Macro

The `should-not>` macro is a short-hand method for `(should-not #'> ...)`.

Syntax

`(should-not> &rest` *`arguments`*`)`

Arguments and Values

arguments These are the arguments to >.

Examples

```
(should-not> 12 13) ; Passes
(should-not> 13 12) ; Fails
(should-not> 12 12) ; Passes
```

1.1.19 The `should<=` Macro

The `should<=` macro is a short-hand method for `(should #'<= ...)`.

Syntax

`(should<= &rest` *`arguments`*`)`

Arguments and Values

arguments These are the arguments to <=.

Examples

```
(should<= 12 13) ; Passes
(should<= 13 12) ; Fails
(should<= 12 12) ; Passes
```

1.1.20 The `should-not<=` Macro

The `should-not<=` macro is a short-hand method for `(should-not #'<= ...)`.

Syntax

`(should-not<= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `<=`.

Examples

```
(should-not<= 12 13) ; Fails
(should-not<= 13 12) ; Passes
(should-not<= 12 12) ; Fails
```

1.1.21 The `should>=` Macro

The `should>=` macro is a short-hand method for `(should #'>= ...)`.

Syntax

`(should>= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `>=`.

Examples

```
(should>= 12 13) ; Fails
(should>= 13 12) ; Passes
(should>= 12 12) ; Passes
```

1.1.22 The `should-not>=` Macro

The `should-not>=` macro is a short-hand method for `(should-not #'>= ...)`.

Syntax

`(should-not>= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `>=`.

Examples

```
(should-not>= 12 13) ; Passes
(should-not>= 13 12) ; Fails
(should-not>= 12 12) ; Fails
```

1.1.23 The `should-eq` Macro

The `should-eq` macro is a short-hand method for `(should #'eq ...)`.

Syntax

`(should-eq &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `eq`.

Examples

```
(should-eq 12 12)       ; Probably passes
(should-eq 13 12)       ; Fails
(should-eq "foo" "foo") ; May pass, may fail.
```

1.1.24 The `should-not-eq` Macro

The `should-not-eq` macro is a short-hand method for `(should-not #'eq ...)`.

Syntax

`(should-not-eq &rest` *`arguments`*`)`

Arguments and Values

arguments These are the arguments to `eq`.

Examples

```
(should-not-eq 12 12)       ; Probably fails
(should-not-eq 13 12)       ; Passes
(should-not-eq "foo" "foo") ; May pass, may fail.
```

1.1.25 The `should-eql` Macro

The `should-eql` macro is a short-hand method for `(should #'eql ...)`.

Syntax

`(should-eql &rest` *arguments*`)`

Arguments and Values

arguments These are the arguments to `eql`.

Examples

```
(should-eql 12 12)       ; Passes
(should-eql 13 12)       ; Fails
(should-eql "foo" "foo") ; May pass, may fail.
```

1.1.26 The `should-not-eql` Macro

The `should-not-eql` macro is a short-hand method for `(should-not #'eql ...)`.

Syntax

`(should-not-eql &rest` *arguments*`)`

Arguments and Values

arguments These are the arguments to `eql`.

Examples

```
(should-not-eql 12 12)       ; Fails
(should-not-eql 13 12)       ; Passes
(should-not-eql "foo" "foo") ; May pass, may fail.
```

1.1.27 The `should-equal` Macro

The `should-equal` macro is a short-hand method for `(should #'equal ...)`.

Syntax

`(should-equal &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `equal`.

Examples

```
(should-equal 12 12)       ; Passes
(should-equal 13 12)       ; Fails
(should-equal "foo" "foo") ; Passes
(should-equal "FOO" "foo") ; Fails
```

1.1.28 The `should-not-equal` Macro

The `should-not-equal` macro is a short-hand method for `(should-not #'equal ...)`.

Syntax

`(should-not-equal &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `equal`.

Examples

```
(should-not-equal 12 12)       ; Passes
(should-not-equal 13 12)       ; Fails
(should-not-equal "foo" "foo") ; Fails
(should-not-equal "FOO" "foo") ; Passes
```

1.1.29 The `should-equalp` Macro

The `should-equalp` macro is a short-hand method for `(should #'equalp ...)`.

Syntax

`(should-equalp &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `equalp`.

Examples

```
(should-equalp 12 12)       ; Passes
(should-equalp 13 12)       ; Fails
(should-equalp "foo" "foo") ; Passes
(should-equalp "FOO" "foo") ; Passes
```

1.1.30 The `should-not-equalp` Macro

The `should-not-equalp` macro is a short-hand method for `(should-not #'equalp ...)`.

Syntax

`(should-not-equalp &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `equalp`.

Examples

```
(should-not-equalp 12 12)       ; Passes
(should-not-equalp 13 12)       ; Fails
(should-not-equalp "foo" "foo") ; Passes
(should-not-equalp "FOO" "foo") ; Fails
```

1.1.31 The `should-string=` Macro

The `should-string=` macro is a short-hand method for `(should #'string= ...)`.

Syntax

`(should-string= &rest arguments)`

Arguments and Values

arguments These are the arguments to `string=`.

Examples

```
(should-string= "foo" "foo") ; Passes
(should-string= "FOO" "foo") ; Fails
```

1.1.32 The `should-not-string=` Macro

The `should-not-string=` macro is a short-hand method for `(should-not #'string= ...)`.

Syntax

`(should-not-string= &rest arguments)`

Arguments and Values

arguments These are the arguments to `string=`.

Examples

```
(should-not-string= "foo" "foo") ; Fails
(should-not-string= "FOO" "foo") ; Passes
```

1.1.33 The `should-string/=` Macro

The `should-string/=` macro is a short-hand method for `(should #'string/= ...)`.

Syntax

`(should-string/= &rest arguments)`

Arguments and Values

arguments These are the arguments to `string/=`.

Examples

```
(should-string/= "foo" "foo") ; Fails
(should-string/= "FOO" "foo") ; Passes
```

1.1.34 The `should-not-string/=` Macro

The `should-not-string/=` macro is a short-hand method for `(should-not #'string/= ...)`.

Syntax

`(should-not-string/= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string/=`.

Examples

```
(should-not-string/= "foo" "foo") ; Passes
(should-not-string/= "FOO" "foo") ; Fails
```

1.1.35 The `should-string<` Macro

The `should-string<` macro is a short-hand method for `(should #'string< ...)`.

Syntax

`(should-string< &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string<`.

Examples

```
(should-string< "foo" "f")      ; Fails
(should-string< "foo" "foo")    ; Fails
(should-string< "foo" "FOOBAR") ; Fails
(should-string< "foo" "foobar") ; Passes
```

1.1.36 The `should-not-string<` Macro

The `should-not-string<` macro is a short-hand method for `(should-not #'string< ...)`.

Syntax

`(should-not-string< &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string<`.

Examples

```
(should-not-string< "foo" "f")      ; Passes
(should-not-string< "foo" "foo")    ; Passes
(should-not-string< "foo" "foobar") ; Fails
```

1.1.37 The `should-string>` Macro

The `should-string>` macro is a short-hand method for `(should #'string> ...)`.

Syntax

`(should-string> &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string>`.

Examples

```
(should-string> "foo" "f")      ; Passes
(should-string> "foo" "foo")    ; Fails
(should-string> "foo" "FOO")    ; Passes
(should-string> "foo" "foobar") ; Fails
```

1.1.38 The `should-not-string>` Macro

The `should-not-string>` macro is a short-hand method for `(should-not #'string> ...)`.

Syntax

(should-not-string> &rest *arguments*)

Arguments and Values

arguments These are the arguments to string>.

Examples

```
(should-not-string> "foo" "f")      ; Fails
(should-not-string> "foo" "foo")    ; Passes
(should-not-string> "foo" "foobar") ; Passes
```

1.1.39 The should-string<= Macro

The should-string<= macro is a short-hand method for (should #'string<= ...).

Syntax

(should-string<= &rest *arguments*)

Arguments and Values

arguments These are the arguments to string<=.

Examples

```
(should-string<= "foo" "f")      ; Fails
(should-string<= "foo" "foo")    ; Passes
(should-string<= "foo" "foobar") ; Passes
```

1.1.40 The should-not-string<= Macro

The should-not-string<= macro is a short-hand method for (should-not #'string<= ..

Syntax

(should-not-string<= &rest *arguments*)

Arguments and Values

arguments These are the arguments to string<=.

Examples

```
(should-not-string<= "foo" "f")      ; Passes
(should-not-string<= "foo" "foo")    ; Fails
(should-not-string<= "foo" "foobar") ; Fails
```

1.1.41 The `should-string>=` Macro

The `should-string>=` macro is a short-hand method for `(should #'string>= ...)`.

Syntax

`(should-string>= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string>=`.

Examples

```
(should-string>= "foo" "f")      ; Passes
(should-string>= "foo" "foo")    ; Passes
(should-string>= "foo" "foobar") ; Fails
```

1.1.42 The `should-not-string>=` Macro

The `should-not-string>=` macro is a short-hand method for `(should-not #'string>= ...)`.

Syntax

`(should-not-string>= &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string>=`.

Examples

```
(should-not-string>= "foo" "f")      ; Fails
(should-not-string>= "foo" "foo")    ; Fails
(should-not-string>= "foo" "foobar") ; Passes
```

1.1.43 The `should-string-equal` Macro

The `should-string-equal` macro is a short-hand method for `(should #'string-equal ...)`.

Syntax

`(should-string-equal &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string-equal`.

Examples

```
(should-string-equal "foo" "foo")    ; Passes
(should-string-equal "FOO" "foo")    ; Passes
(should-string-equal "foo" "foobar") ; Fails
```

1.1.44 The `should-not-string-equal` Macro

The `should-not-string-equal` macro is a short-hand method for `(should-not #'string-equal ...)`.

Syntax

`(should-not-string-equal &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string-equal`.

Examples

```
(should-not-string-equal "foo" "foo")    ; Fails
(should-not-string-equal "FOO" "foo")    ; Fails
(should-not-string-equal "foo" "foobar") ; Passes
```

1.1.45 The `should-string-not-equal` Macro

The `should-string-not-equal` macro is a short-hand method for `(should #'string-not-equal ...)`.

Syntax

`(should-string-not-equal &rest` *`arguments`*`)`

Arguments and Values

arguments These are the arguments to `string-not-equal`.

Examples

```
(should-string-not-equal "foo" "foo")    ; Fails
(should-string-not-equal "FOO" "foo")    ; Fails
(should-string-not-equal "foo" "foobar") ; Passes
```

1.1.46 The `should-not-string-not-equal` Macro

The `should-not-string-not-equal` macro is a short-hand method for `(should-not #'string-not-equal ...)`.

Syntax

`(should-not-string-not-equal &rest` *arguments*`)`

Arguments and Values

arguments These are the arguments to `string-not-equal`.

Examples

```
(should-not-string-not-equal "foo" "foo")    ; Passes
(should-not-string-not-equal "FOO" "foo")    ; Passes
(should-not-string-not-equal "foo" "foobar") ; Fails
```

1.1.47 The `should-string-lessp` Macro

The `should-string-lessp` macro is a short-hand method for `(should #'string-lessp ...)`.

Syntax

`(should-string-lessp &rest` *arguments*`)`

Arguments and Values

arguments These are the arguments to `string-lessp`.

Examples

```
(should-string-lessp "foo" "f")      ; Fails
(should-string-lessp "foo" "foo")    ; Fails
(should-string-lessp "foo" "FOOBAR") ; Passes
(should-string-lessp "foo" "foobar") ; Passes
```

1.1.48 The `should-not-string-lessp` Macro

The `should-not-string-lessp` macro is a short-hand method for `(should-not #'string-lessp ...)`.

Syntax

`(should-not-string-lessp &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string-lessp`.

Examples

```
(should-not-string-lessp "foo" "f")      ; Passes
(should-not-string-lessp "foo" "foo")    ; Passes
(should-not-string-lessp "foo" "FOOBAR") ; Fails
(should-not-string-lessp "foo" "foobar") ; Fails
```

1.1.49 The `should-string-greaterp` Macro

The `should-string-greaterp` macro is a short-hand method for `(should #'string-greaterp ...)`.

Syntax

`(should-string-greaterp &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string-greaterp`.

Examples

```
(should-string-greaterp "foo" "f")      ; Passes
(should-string-greaterp "foo" "foo")    ; Fails
(should-string-greaterp "foo" "FOO")    ; Fails
(should-string-greaterp "foo" "foobar") ; Fails
```

1.1.50 The `should-not-string-greaterp` Macro

The `should-not-string-greaterp` macro is a short-hand method for `(should-not #'string-greaterp ...)`.

Syntax

`(should-not-string-greaterp &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string-greaterp`.

Examples

```
(should-not-string-greaterp "foo" "f")      ; Fails
(should-not-string-greaterp "foo" "foo")    ; Passes
(should-not-string-greaterp "foo" "FOO")    ; Passes
(should-not-string-greaterp "foo" "foobar") ; Passes
```

1.1.51 The `should-string-not-greaterp` Macro

The `should-string-not-greaterp` macro is a short-hand method for `(should #'string-not-greaterp ...)`.

Syntax

`(should-string-not-greaterp &rest` *`arguments`*`)`

Arguments and Values

`arguments` These are the arguments to `string-not-greaterp`.

Examples

```
(should-string-not-greaterp "foo" "f")      ; Fails
(should-string-not-greaterp "foo" "foo")    ; Passes
(should-string-not-greaterp "foo" "FOO")    ; Passes
(should-string-not-greaterp "foo" "foobar") ; Passes
```

1.1.52 The `should-not-string-not-greaterp` Macro

The `should-not-string-not-greaterp` macro is a short-hand method for `(should-not #'string-not-greaterp ...)`.

Syntax

```
(should-not-string-not-greaterp &rest arguments)
```

Arguments and Values

arguments These are the arguments to `string-not-greaterp`.

Examples

```
(should-not-string-not-greaterp "foo" "f")      ; Passes
(should-not-string-not-greaterp "foo" "foo")    ; Fails
(should-not-string-not-greaterp "foo" "FOO")    ; Fails
(should-not-string-not-greaterp "foo" "foobar") ; Fails
```

1.1.53 The `should-string-not-lessp` Macro

The `should-string-not-lessp` macro is a short-hand method for `(should #'string-not-lessp ...)`.

Syntax

```
(should-string-not-lessp &rest arguments)
```

Arguments and Values

arguments These are the arguments to `string-not-lessp`.

Examples

```
(should-string-not-lessp "foo" "f")      ; Passes
(should-string-not-lessp "foo" "foo")    ; Passes
(should-string-not-lessp "foo" "FOOBAR") ; Fails
(should-string-not-lessp "foo" "foobar") ; Fails
```

1.1.54 The `should-not-string-not-lessp` Macro

The `should-not-string-not-lessp` macro is a short-hand method for `(should-not #'string-not-lessp ...)`.

Syntax

```
(should-not-string-not-lessp &rest arguments)
```

Arguments and Values

arguments These are the arguments to `string-not-lessp`.

Examples

```
(should-not-string-not-lessp "foo" "f")      ; Fails
(should-not-string-not-lessp "foo" "foo")    ; Fails
(should-not-string-not-lessp "foo" "FOOBAR") ; Passes
(should-not-string-not-lessp "foo" "foobar") ; Passes
```

Chapter 2

The `sigma/control` Package

The `sigma/control` package contains code for basic program control systems. These are mostly basic macros to add more complicated looping, conditionals, or similar. These are typically extensions to Common Lisp that are inspired by other programming languages. Thanks to the power of Common Lisp and its macro system, we can typically implement most features of any other language with little trouble.

2.1 Macros

2.1.1 The `aif` Macro

The `aif` macro is an anaphoric variation of the built-in `if` control structure. This is based on [1, p. 190]. The basic idea is to provide an anaphor (such as pronouns in English) for the conditional so that it can easily be referred to within the body of the conditional expression. The most natural pronoun in the English language for a thing is "it", so that is what is used. If you need or want to use a different anaphor, use `a?if`. The most common use of `aif` is for when you want to do some additional computation with some time-consuming calculation, but only if it returned successfully.

Syntax

`(aif` *`conditional t-action`* `&optional` *`nil-action`*`)`

Arguments and Values

conditional The boolean conditional to select between the *t-action* and the *nil-action*.

t-action The action to evaluate if the *conditional* evaluates as true.

nil-action The action to evaluate if the *conditional* evaluates as nil.

Examples

```
(aif (big-long-calculation)
     (foo it)
     (format t "The␣big-long-calculation␣failed!~%"))
```

This is similar to the following, but with less typing:

```
(let ((it (big-long-calculation)))
  (if it
      (foo it)
\      (format t "The␣big-long-calculation␣failed!~%")))
```

Or say you need to get a user name from a database call, which might be slow.

```
(aif (get-user-name)
     (format -t "Hello,␣~A!~%" it)
     (format -t "You␣aren't␣logged␣in,␣go␣away!~%"))
```

2.1.2 The a?if Macro

The a?if macro is a variation of aif that allows for the specification of the anaphor to use, instead of being restricted to just it, the default with aif. This is most often useful when you need to nest calls to anaphoric macros.

Syntax

(a?if *anaphor conditional t-action* &optional *nil-action*)

Arguments and Values

anaphor The result of the *conditional* will be stored in the variable specified as the anaphor.

conditional The boolean conditional to select between the *t-action* and the *nil-action*.

t-action The action to evaluate if the *conditional* evaluates as true.

nil-action The action to evaluate if the *conditional* evaluates as nil.

Examples

```
(a?if foo 'outer
  (a?if bar 'inner
    `(,foo ,bar))) ; Returns '(outer inner)
```

2.1.3 The aand Macro

The aand macro is an anaphoric variation of the built-in and. This is based on [1, p. 191]. It works in a similar manner to aif, defining it as the current argument for use in the next argument, reassigning it with each argument.

Syntax

```
(aand &rest arguments)
```

Examples

```
(aand 2         ; Sets 'it' to 2.
      (* 3 it)  ; Sets 'it' to 6.
      (* 4 it)) ; Returns 24.
```

2.1.4 The a?and Macro

The a?and macro is a variant of aand that allows for the specification of the anaphor to use, instead of being restricted to just it, the default with aand. This is most often useful when you need to nest calls to anaphoric macros.

Examples

```
(a?and foo 12 (* 2 foo) (* 3 foo)) ; Returns 72.

(a?and foo 1 2 3 'outer
  (a?and bar 4 5 6 'inner `(,foo ,bar))) ; Returns '(outer inner)
```

2.1.5 The `alambda` Macro

The `alambda` macro is an anaphoric variant of the built-in `lambda`. This is based on [1, p. 193]. It works in a similar manner to `aif` and `aand`, except it defines `self` instead of `it` as the default anaphor. This is useful so that you can write recursive lambdas.

```
(funcall (alambda (x) ; Simple recursive factorial example.
           (if (<= x 0)
               1
               (* x (self (1- x)))))
         10))) ; Calculates 10!, inefficiently.
```

2.1.6 The `a?lambda` Macro

The `a?lambda` macro is a variant of `alambda` that allows you to specify the anaphor to use, instead of just the default of `it`.

```
(funcall (a?lambda ! (x) ; Simple recursive factorial example.
           (if (<= x 0)
               1
               (* x (! (1- x)))))
         10))) ; Calculates 10!, inefficiently.
```

2.1.7 The `ablock` Macro

The `ablock` macro is an anaphoric variant of the built-in `block`. This is based on [1, p. 193]. It works in a similar manner to `aand`, defining the anaphor `it` for each argument to the block.

Examples

```
(let (w x y z)
   (ablock b
           (setf w 7)
           (setf x (* 2 it)) ; Twice w, 14.
           (setf y (* 3 it)) ; Thrice x, 42.
           (return-from b)   ; Leave the block.
           (setf z 123))     ; Never happens.
   (list w x y z))           ; Returns '(7 14 42 nil)
```

2.1.8 The `a?block` Macro

The `a?block` macro is an anaphoric variant of `ablock` that allows you to specify the anaphor to use, instead of just the default of `it`.

Examples

```
(let (w x y z)
   (a?block b foo
            (setf w 7)
            (setf x (* 2 foo)) ; Twice w, 14.
            (setf y (* 3 foo)) ; Thrice x, 42.
            (return-from b)    ; Leave the block.
            (setf z 123))      ; Never happens.
   (list w x y z))             ; Returns '(7 14 42 nil)
```

2.1.9 The `acond` Macro

The `acond` macro is an anaphoric variant of the built-in `cond`. This is based on [1, p. 191]. It works in a similar manner to `aand`, defining the anaphor `it` for each argument to the conditional.

Examples

```
(let (a b (c 3))
  (acond (a it)          ; No.
         (b it)          ; No.
         (c (* 4 it)))) ; Yes, returns 12 = 4*3, the value of c.
```

2.1.10 The `a?cond` Macro

The `a?cond` macro is an anaphoric variant of `acond` that allows you to specify the anaphor to use, instead of just the default of `it`.

Examples

```
(let (a b (c 3))
  (a?cond foo
          (a foo)          ; No.
          (b foo)          ; No.
          (c (* 4 foo)))) ; Yes, returns 12 = 4*3, the value of c.
```

2.1.11 The `awhen` Macro

The `awhen` macro is an anaphoric variant of the built-in `when`. This is based on [1, p. 191]. It works in a similar manner to `aif`, defining `it` as the default anaphor. This is useful when the conditional is the result of a complicated computation, so you don't have to compute it twice or wrap the computation in a let block yourself.

Syntax

(awhen *conditional* &body *body*)

Examples

```
(awhen (get-user-name)
  (do-something-with-name it)
  (do-more-stuff)
  (format -t "Hello,␣~A!~%" it))
```

2.1.12 The a?when Macro

The a?when macro is similar to the awhen, except that it allows you to specify the anaphor to use, instead of just the default of it.

Syntax

(a?when *anaphor conditional* &body *body*)

Examples

```
(a?when user (get-user-name)
  (do-something-with-name user)
  (do-more-stuff)
  (format -t "Hello,␣~A!~%" user))
```

2.1.13 The awhile Macro

The awhile macro is an anaphoric variant of while. This is based on [1, p. 191]. This is useful if you need to consume input repeatedly for all input. It returns the value of the last form in the body from the last iteration, or nil if the body never runs.

Syntax

(awhile *expression* &body *body*)

Examples

```
(awhile (get-input)
  (do-something it)) ; Operate on input for all input.

(let ((forward '(1 2 3))
      (backward nil))
```

```
(awhile (pop forward)
  (push it backward))) ; Returns (3 2 1)
```

2.1.14 The `a?while` Macro

The `a?while` macro is a variant of `awhile` that allows you to specify the anaphor to use, instead of just the default `it`. It returns the value of the last form in the body from the last iteration, or `nil` if the body never runs.

Syntax

`(a?while` *anaphor expression* `&body` *body*`)`

Examples

```
(a?while input (get-input)
  (do-something input)) ; Operate on input for all input.
```

2.1.15 The `defconstant-once` Macro

The `defconstant-once` macro defines a constant using `defconstant`, but only if it is not already bound. It expands to a top-level `defconstant` inside `eval-when` so the compiler can see the name when compiling later forms in the same file; on reload the value form returns the existing binding so the constant is not redefined inconsistently. This is particularly useful for large immutable datasets so that reloading a file with ASDF does not produce redefinition warnings.

Syntax

`(defconstant-once` *name value* `&optional` *docstring*`)`

Arguments and Values

name The name of the constant (conventionally using `+plus-signs+`).

value The value of the constant. This expression is evaluated only the first time the constant is defined.

docstring An optional documentation string for the constant.

Examples

```
(defconstant-once +reward-a1-stats+
  '((697 46.467 1345840.30 0.49023)
    ...)
  "Statistics␣from␣reward-a1␣runs.")
```

This macro is defined in the `sigma/control` package.

2.1.16 The `deletef` Macro

The `deletef` macro deletes `item` from `sequence` in-place.

Syntax

`(deletef` *item sequence* `&rest` *rest*`)`

Examples

```
(let ((men '(good bad ugly)))
  (deletef 'bad men)
  (deletef 'ugly men)
  men) ; Only the good is left.
```

2.1.17 The `do-while` Macro

The `do-while` macro operates like a `do {BODY} while (CONDITIONAL)` in the C programming language. The body always runs at least once. The macro returns the value of the last form in the body from the last iteration.

Syntax

`(do-while` *conditional* `&body` *body*`)`

Examples

```
(let ((t-minus 10))
  (do-while (<= 0 t-minus)
    (format t "~A␣...␣" t-minus)
    (decf t-minus)))
(format t "Liftoff!~%")
```

2.1.18 The `do-until` Macro

The `do-until` macro operates like a `do {`*body*`} while (! `*conditional*`)` in the C programming language. The body always runs at least once. The macro returns the value of the last form in the body from the last iteration.

Syntax

`(do-until `*conditional* `&body `*body*`)`

Examples

```
(let ((t-minus 10))
  (do-until (< t-minus 0)
    (format t "~A␣...␣" t-minus)
    (decf t-minus)))
(format t "Liftoff!~%")
```

2.1.19 The `fop` Macro

`fop` is like the `opf` macro, but as a post-assignment variant. The difference is similar to the difference between `x++` and `++x` in the C Programming Language, with opf being like `++x` and fop being like `x++`.

Syntax

`(fop `*operator variable* `&rest `*arguments*`)`

Examples

```
(let ((x 10))
  (while (<= 0 x)
    (format t "~A␣...␣" (fop #'- x 1))))
(format t "Liftoff!~%")
```

2.1.20 The `for` Macro

A `for` macro, much like the `for` in the C programming language.

Syntax

`(for `*initial conditional step-action* `&body `*body*`)`

Examples

```
(for ((i 0))
     (< i 10)
     (incf i)
  (format t \"~%~A\" i))
```

2.1.21 The `forever` Macro

The `forever` macro is just a way to say `(while t ...)` with a bit of added expressiveness and explicitness.

Examples

```
(forever (let ((in (read)))
           (if (eq in 'quit))
               (format t "I can't let you do that, Dave.")
               (format t "You entered ~A" in)))
```

2.1.22 The `multicond` Macro

The `multicond` macro is much like `cond`, but where multiple clauses may be evaluated.

Examples

```
(let ((x 12))
  (multicond ((= x 12) ; This will evaluate.
              (format t "X is 12!  My favorite number!~%"))
             ((< x 100) ; This will evaluate also.
              (format t "X is small.~%"))
             ((< x 0) ; But this one won't.
              (format t "X is negative.~%"))))
```

2.1.23 The `opf` Macro

The `opf` macro is a generic operate-and-store, along the lines of `incf` and `decf`, but allowing for any operation.

Syntax

(opf *operator variable* &rest *arguments*)

Examples

```
;;; Prints 1 ... 2 ... 4 ... 8 ... ··· ... 65535 ... that's it!
(let ((x 1))
  (while (<= x (expt 2 16))
    (format t "~A␣...␣" x)
    (opf #'* x 2)))
(format t "␣that's␣it!~%")
```

2.1.24 The `swap` Macro

This is a simple `swap` macro. The values of the first and second form are swapped with each other.

Syntax

`(swap` *x y*`)`

Examples

```
(let ((first "the␣first")
      (last "the␣last"))
  (swap first last)
  `(,first ,last)) ; Returns '("the last" "the first")
```

2.1.25 The `swap-unless` Macro

This macro calls `swap` unless the predicate evaluates to true.

Syntax

`(swap-unless` *predicate x y*`)`

Examples

```
;;; make smaller and larger in the correct order.
(let ((smaller 12)
      (larger 266))
  (swap-unless #'<= smaller larger))
```

2.1.26 The `swap-when` Macro

This macro calls `swap` when the predicate evaluates to true.

Syntax

(swap-when *predicate x y*)

Examples

```
;;; make smaller and larger in the correct order.
(let ((smaller 12)
      (larger 6))
  (swap-when #'> smaller larger))
```

2.1.27 The `until` Macro

The `until` macro is similar to the while loop in C, but with a negated conditional. It returns the value of the last form in the body from the last iteration, or `nil` if the body never runs.

Syntax

(until *conditional* &body *body*)

Examples

```
(let ((x 10))
  (until (< x 0)
    (format t "~A␣...␣" x)
    (decf x))
  (format t "Liftoff!~%"))

(let ((x 0))
  (until (<= 10 x)
    (incf x))) ; Returns 10
```

2.1.28 The `while` Macro

This `while` macro is similar to the while loop in C. It returns the value of the last form in the body from the last iteration, or `nil` if the body never runs.

Syntax

(while *conditional* &body *body*)

Examples

```
(let ((x 10))
  (while (<= 0 x)
    (format t "~A␣...␣" x)
    (decf x))
  (format t "Liftoff!~%"))

(let ((x 0))
  (while (< x 3)
    (incf x)
    (* x 10))) ; Returns 30
```

2.1.29 The `macro-alias` Macro

The `macro-alias` macro produces one or more alternate names for a macro by copying its macro function.

Syntax

`(macro-alias` *`macro`* `&rest` *`aliases`*`)`

Examples

```
(macro-alias when whenever)
```

2.2 Functions

2.2.1 The `compose` Function

The `compose` function composes a single function from a list of several functions such that the new function is equivalent to calling the functions in succession. This is based upon a `compose` function in [2], which is based upon the `compose` function from Dylan.

Syntax

`(compose &rest` *`functions`*`)`

Examples

We want to calculate:

$$\sin(\cos(\tan(\pi))) \approx 0.841{,}470{,}984{,}807{,}896{,}5$$

```
(funcall (compose #'sin #'cos #'tan) pi)
(sin (cos (tan pi))) ; This is the same.
```

2.2.2 The `juxtapose` Function

The `juxtapose` function takes any number of functions and returns a new function that is their juxtaposition, as in Clojure's `juxt`. The returned function applies each of those functions to its arguments and returns a list of the results in order. Unlike `compose`, which pipes a value through functions in series, `juxtapose` fans out the same arguments to each function independently. The name `juxt` is a `function-alias` for `juxtapose`.

Syntax

```
(juxtapose &rest functions)
  (juxt &rest functions)
```

Examples

```
(funcall (juxtapose #'1+ #'1-) 10)      ; Returns (11 9)
(funcall (juxt #'car #'cdr) '(a b c))   ; Returns (A (B C))
(funcall (juxtapose #'+ #'*) 2 3 4)     ; Returns (9 24)
```

2.2.3 The `thread-first` Macro

The `thread-first` macro (Clojure's `->`) inserts an expression as the first argument of a sequence of forms, nesting left-to-right. That is,

```
(thread-first x (foo y) (bar z) (baz w))
```

expands to

```
(baz (bar (foo x y) z) w)
```

instead of writing the nested form by hand. A non-list form `f` is treated as `(f)`. With no forms, the macro expands to the initial expression alone. The name `->` is a `macro-alias` for `thread-first`.

Syntax

```
(thread-first x &rest forms)
  (-> x &rest forms)
```

Examples

```
(-> 2 (* 3) (+ 3))              ; Returns 9
(thread-first 5 1+)             ; Returns 6
(-> x (foo y) (bar z) (baz w))  ; => (baz (bar (foo x y) z) w)
```

2.2.4 The `thread-last` Macro

The `thread-last` macro (Clojure's `->>`) inserts an expression as the *last* argument of a sequence of forms, nesting left-to-right. That is,

```
(thread-last x (foo y) (bar z) (baz w))
```

expands to

```
(baz w (bar z (foo y x)))
```

A non-list form `f` is treated as `(f)`. With no forms, the macro expands to the initial expression alone. The name `->>` is a `macro-alias` for `thread-last`.

Syntax

```
(thread-last x &rest forms)
  (->> x &rest forms)
```

Examples

```
(->> '(1 2 3 4 5)
     (mapcar #'1+)
     (remove-if-not #'evenp))   ; Returns (2 4 6)
(->> x (foo y) (bar z) (baz w)) ; => (baz w (bar z (foo y x)))
```

2.2.5 The `conjoin` Function

The `conjoin` function takes in one or more predicates, and returns a predicate that returns true whenever all of the predicates return true. This is from [2] and is based upon the `conjoin` function from Dylan.

Syntax

```
(conjoin predicate &rest predicates)
```

Examples

```
;;; Returns '(6 12 18 24 30 36 42 48 54 60 66 72 78 84 90 96).
(remove-if-not #'identity
  (flet ((mod-2? (i) (zerop (mod i 2)))
         (mod-3? (i) (zerop (mod i 3))))
    (loop for i from 1 to 100 collect
          (when (funcall (conjoin #'mod-2? #'mod-3?) i)
                i))))
```

2.2.6 The `curry` Function

The `curry` function takes in a function and some of its arguments, and returns a function that expects the rest of the required arguments. This is from [2] and is based upon the `curry` function from Dylan.

Syntax

`(curry` *function* `&rest` *arguments*`)`

Examples

```
(let ((x 100)
      (f (curry #'+ x)))
  (loop for i from 1 to 10 collect
        (funcall f i))) ; Returns '(101 102 103 ... 110)
```

2.2.7 The `disjoin` Function

The `disjoin` function takes in one or more predicates, and returns a predicate that returns true whenever any of the predicates return true. This is from [2] and is based upon the disjoin function from Dylan.

Syntax

`(disjoin` *predicate* `&rest` *predicates*`)`

Examples

```
;;; Returns  '(#\1 #\2 #\3 #\a #\b #\c NIL     NIL)
(let ((chars '(#\1 #\2 #\3 #\a #\b #\c #\Space #\Newline)))
  (mapcar (lambda (c)
            (when (funcall (disjoin #'alpha-char-p #'digit-char-p)
                           c)
                  c))
          chars))
```

2.2.8 The `function-alias` Macro

The `function-alias` macro produces one or more aliases (alternate names) for a function. Each name in `aliases` is given the same `fdefinition` as `function`.

Syntax

`(function-alias` *`function`* `&rest` *`aliases`*`)`

Examples

```
(function-alias 'that-guy-doesnt-know-when-to-stop-typing 'shorter)
(function-alias 'long-name 'short-a 'short-b)
```

2.2.9 The `function-aliases` Macro

The `function-aliases` macro is an alias for `function-alias`.

2.2.10 The `function-alias-as-a-function` Function

The `function-alias-as-a-function` function is the older functional form of `function-alias`. It assigns the function definition of `function` to each name in `aliases` by modifying `fdefinition`. Prefer the `function-alias` macro in new code.

Syntax

`(function-alias-as-a-function` *`function`* `&rest` *`aliases`*`)`

Examples

```
(function-alias-as-a-function 'long-name 'short)
```

2.2.11 The `function-aliases-as-a-function` Function

The `function-aliases-as-a-function` function is an alias for `function-alias-as-a-function`.

2.2.12 The `operator-to-function` Function

The `operator-to-function` function takes in any symbol and makes an evaluatable function out of it. The principle purpose for this is so that we can treat macros and other non-function things like a function, for using them with `mapcar` or similar.

Known Issues

[Issue #8]

Syntax

`(operator-to-function` *operator*`)`

Examples

```
;;; In case you don't like (setf a 1 b 2 c 3).
(mapcar (operator-to-function 'setf)
        '(a b c)
        '(1 2 3))
```

2.2.13 The `rcompose` Function

The `rcompose` function is a reversed variant of the `compose` function.

Syntax

`(rcompose &rest` *functions*`)`

Examples

We want to calculate:

$$\tan\left(\cos\left(\sin\left(\pi\right)\right)\right) \approx 1.557{,}407{,}724{,}654{,}902{,}3$$

```
(funcall (rcompose #'sin #'cos #'tan) pi)
(tan (cos (sin pi))) ; This is the same.
```

2.2.14 The `rcurry` Function

This function takes in a function and some of its ending arguments, and returns a function that expects the rest of the required arguments. This is from [2] and is based upon the `rcurry` function from Dylan.

Syntax

`(rcurry` *function* `&rest` *arguments*`)`

Examples

```
(let ((x 100)
      (f (rcurry #'- x)))
  (loop for i from 1 to 10 collect
        (funcall f i))) ; Returns '(-99 -98 -97 ... -90)
```

2.2.15 The `unimplemented` Function

This is a convenience function that merely raises an error. It is for code that is yet to be written.

Syntax

```
(unimplemented)
```

Examples

```
(defun turing-test-solver ()
  (unimplemented)) ; TODO: figure out how to program this.
```

2.3 Symbols

2.3.1 The `it` Variable

The symbol `it` is the default anaphor used by macros such as `aif`, `aand`, `awhen`, `awhile`, `acond`, and `ablock`. It is exported so that client code may refer to the anaphor without package qualification. It is not a global variable; each anaphoric macro binds it locally.

2.3.2 The `self` Variable

The symbol `self` is the default anaphor used by `alambda` for recursive calls. It is exported so that client code may refer to it without package qualification. It is not a global variable; `alambda` binds it locally as a local function.

2.4 Generics

2.4.1 The `duplicate` Generic

The `duplicate` generic is to provide a deep copy facility for any of your objects. If you define a class and want a deep copy facility for it, implement a version of `duplicate` that is correct for it. This library provides

methods for lists (deeply mapped), arrays (deeply copied element-wise), numbers, symbols, and functions (the last three returned unchanged, being immutable or not meaningfully copyable).

Syntax

```
(duplicate item)
```

Examples

```
(duplicate '(1 2 (3 4))) ; Returns a fresh tree equal to the original
(duplicate 42)           ; Returns 42
```

Chapter 3

The `sigma/hash` Package

3.1 Macros

3.1.1 The `sethash` Macro

The `sethash` macro is a shortcut for `setf gethash`.

Syntax

`(sethash` *`key hash-table value`*`)`

Arguments and Values

`key` The key to set in the hash table.

`hash-table` The hash table we are modifying.

`value` The value to store under that key.

Returns

The new *`value`* for the key.

3.1.2 The `set-partition` Macro

The `set-partition` macro is a variant of `sethash` that assumes the hash entries are all sequences, allowing for multiple results per key. When you call `gethash` on the hashmap you will get back a sequence of all the entries for that key.

Syntax

`(set-partition` *`key hash-table value`*`)`

Arguments and Values

key The key to add to in the hash table.

hash-table The partition, a hash table, we are modifying.

value The value to add to the hash table key.

Returns

The new set of values for key, a sequence containing your newly added *value* and any previously added values.

3.2 Functions

3.2.1 The `populate-hash-table` Function

The `populate-hash-table` function makes initial construction of hash tables a lot easier, just taking in key/value pairs as the arguments to the function, and returning a newly-constructed hash table.

Examples

```
(populate-hash-table 'name "Valentinus"
                     'likes '(birds roses)
                     'dislikes '(beheadings epilepsy "false idols")
                     'died 269)
```

3.2.2 The `inchash` Function

The `inchash` function will increment the value in *key* of the *hash*, initializing it to 1 if it isn't currently defined.

3.2.3 The `dechash` Function

The `dechash` function will decrement the value in *key* of the *hash*, initializing it to -1 if it isn't currently defined.

3.2.4 The `gethash-in` Function

The `gethash-in` function works like `gethash`, but allows for multiple keys to be specified at once, to work with nested hash tables.

Syntax

`(gethash-in` *keys hash-table* `&optional` *default*`)`

Arguments and Values

keys A list of objects.

hash-table A hash table.

default An object. The default is nil.

Returns

value An object.

present? A generalized boolean.

Examples

```
(let ((h (make-hash-table)))
  (sethash 'a h 12)
  (gethash-in '(a) h)) ; Returns 12

(let ((h (make-hash-table))
      (i (make-hash-table)))
  (sethash 'b i 123)
  (sethash 'a h i)
  (gethash-in '(a b) h 123)) ; Returns 123
```

3.2.5 The `make-partition` Function

The `make-partition` function is a variant of `make-hash-table` that assumes you're going to use the hash table for partitions with multiple entries per key, not a one-to-one hashmap.

Syntax

`(make-partition)`

3.2.6 The `populate-partition` Function

The `populate-partition` function makes initial construction of a partition a lot easier, just taking in key/value pairs as the arguments to the function, where there can be multiple entries for any key, and returning a newly-constructed partition.

Syntax

`(populate-partition &rest` *pairs*`)`

Examples

```
(populate-partition 'a '(1 2 3)
                    'a 4
                    'a '(5 6 7)
                    'b 8
                    'c 9
                    'c 10
                    'd 11
                    'a 147)
```

Chapter 4

The `sigma/numeric` Package

4.1 Macros

4.1.1 The `+f` Macro

The `+f` macro is an alias for `incf`.

4.1.2 The `-f` Macro

The `-f` macro is an alias for `decf`.

4.1.3 The `*f` Macro

The `*f` macro is an alias for `multf`.

4.1.4 The `/f` Macro

The `/f` macro is an alias for `divf`.

4.1.5 The `divf` Macro

The `divf` macro is divide-and-store, along the lines of `incf` and `decf`, but with division instead. This is similar to `x /= something` in the C programming language.

Syntax

`(divf` *`variable`* `&rest` *`arguments`*`)`

Examples

```
;;; Prints 65536 ... ··· 8 ... 4 ... 2 ... 1 ... 0 ... that's it!
(let ((x (expt 2 16)))
  (while (<= 0 x)
    (format t "~A␣...␣" x)
    (divf x 2)))
(format t "␣that's␣it!~%")
```

4.1.6 The f+ Macro

The f+ macro is similar to incf or +f, but it is a post-increment instead of a pre-increment. That is, f+ works like x++ in C but incf and +f work like ++x in C.

Syntax

(f+ *variable* &rest *addends*)

Examples

```
(let ((x 12))
  (list x (f+ x) x)) ; Returns '(12 12 13).
```

4.1.7 The f- Macro

The f- macro is similar to decf or -f, but it is a post-decrement instead of a pre-decrement. That is, f- works like x-- in C but decf and -f work like --x in C.

Syntax

(f- *variable* &rest *subtrahends*)

Examples

```
(let ((x 12))
  (list x (f- x) x)) ; Returns '(12 12 11).
```

4.1.8 The f* Macro

The f* macro is similar to multf or *f, but it is a post-multiply instead of a pre-multiply. That is, f* works like x++ in C (just for multiplication instead of addition) but multf and *f work like ++x in C (again, just for multiplication instead of addition.)

Syntax

(f* *variable* &rest *multiplicands*)

Examples

```
(let ((x 12))
  (list x (f* x 2) x)) ; Returns '(12 12 24).
```

4.1.9 The f/ Macro

The f/ macro is similar to divf or /f, but it is a post-divide instead of a pre-divide. That is, f/ works like x++ in C (just for division instead of addition) but divf and /f work like ++x in C (again, just for division instead of addition.)

Syntax

(f/ *variable* &rest *divisors*)

Examples

```
(let ((x 12))
  (list x (f/ x 2) x)) ; Returns '(12 12 6).
```

4.1.10 The multf Macro

The multf macro is multiply-and-store, along the lines of incf and decf, but with multiplication instead. This is similar to x *= something* in the C programming language.

Syntax

(multf *variable* &rest *arguments*)

Examples

```
;;; Prints 1 ... 2 ... 4 ... 8 ... ··· ... 65535 ... that's it!
(let ((x 1))
  (while (<= x (expt 2 16))
    (format t "~A ... " x)
    (multf x 2)))
(format t " that's it!~%")
```

4.2 Functions

4.2.1 The `bit?` Function

The `bit?` function returns true if its argument is of type `bit` (that is, either 0 or 1).

Syntax

`(bit?` *b*`)`

Examples

```
(bit? 0)   ; Returns T
(bit? 1)   ; Returns T
(bit? 2)   ; Returns NIL
(bit? 0.5) ; Returns NIL
```

4.2.2 The `choose` Function

The `choose` function computes the binomial coefficient for n and k, typically spoken as *n choose k*, and usually written mathematically as $\binom{n}{k}$.

Syntax

`(choose` *n k*`)`

Arguments and Values

n A positive integer.

k A positive integer such that $0 < k < n$.

Examples

```
(choose 12 2) ; Returns 66
```

4.2.3 The `factorial` Function

The `factorial` function computes $n!$ for positive integers. NB, this is not intelligent, and uses a loop instead of better approaches.

Syntax

`(factorial` *n*`)`

Arguments and Values

n A positive integer.

Examples

```
(factorial 5) ; Returns 120
(factorial 1) ; Returns 1
```

4.2.4 The `fractional-part` Function

The `fractional-part` function returns the fractional part of a number in the form most familiar to computer scientists. It has the useful property that

$$\mathrm{frac}(x) + \mathrm{int}(x) = x,$$

but the result may be negative. See `http://mathworld.wolfram.com/FractionalPart.html`.

Syntax

`(fractional-part` *number*`)`

Examples

```
(fractional-part 10.5)  ; Returns 0.5
(fractional-part 10)    ; Returns 0
(fractional-part -10.5) ; Returns -0.5
(fractional-part -10)   ; Returns 0
```

4.2.5 The `fractional-value` Function

The `fractional-value` function returns the fractional value of a number in the form most familiar to mathematicians. The result is always nonnegative, and forms a sawtooth wave. This is known as `SawtoothWave` in Mathematica. See `http://mathworld.wolfram.com/FractionalPart.html`. The symbol `sawtooth-wave` is an alias for this function.

Syntax

`(fractional-value` *number*`)`
`(sawtooth-wave` *number*`)`

Examples

```
(fractional-value 10.5)  ; Returns 0.5
(fractional-value 10)    ; Returns 0
(fractional-value -10.5) ; Returns 0.5
(fractional-value -10)   ; Returns 0
```

4.2.6 The `integer-range` Function

The `integer-range` function generates lists of integer ranges of the form [*start*, *stop*]. It has three forms.

Syntax

```
(integer-range stop)
  (integer-range start stop)
  (integer-range start stop step)
```

Arguments and Values

stop The inclusive end of the range. When it is the sole argument, the range begins at 0.

start The inclusive start of the range.

step The stride between successive values. When omitted, the step is 1 or −1 according to the direction from start to stop.

Examples

```
(integer-range 10)
;; Returns '(0 1 2 3 4 5 6 7 8 9 10)

(integer-range 5 10)
;; Returns '(5 6 7 8 9 10)

(integer-range 5 10 2)
;; Returns '(5 7 9)

(integer-range -5 0)
;; Returns '(-5 -4 -3 -2 -1 0)

(integer-range 10 5 -1)
;; Returns '(10 9 8 7 6 5)
```

4.2.7 The `nonnegative?` Function

The `nonnegative?` function returns true if its argument is not negative (that is, if $x \geq 0$).

Syntax

```
(nonnegative?  x)
```

Examples

```
(nonnegative? 0)   ; Returns T
(nonnegative? 3.5) ; Returns T
(nonnegative? -1)  ; Returns NIL
```

4.2.8 The `nonnegative-integer?` Function

The `nonnegative-integer?` function returns true if its argument is an integer greater than or equal to 0.

Syntax

```
(nonnegative-integer?  x)
```

Examples

```
(nonnegative-integer? 12)   ; Returns T
(nonnegative-integer? 0)    ; Returns T
(nonnegative-integer? 12.7) ; Returns NIL
(nonnegative-integer? -12)  ; Returns NIL
```

4.2.9 The `positive-integer?` Function

The `positive-integer?` function returns true if its argument is an integer strictly greater than 0.

Syntax

```
(positive-integer?  x)
```

Examples

```
(positive-integer? 12)   ; Returns T
(positive-integer? 0)    ; Returns NIL
(positive-integer? 12.7) ; Returns NIL
(positive-integer? -12)  ; Returns NIL
```

4.2.10 The `product` Function

The `product` function returns the product of the elements of a sequence, optionally transformed by a key function and restricted to a subsequence. An empty range yields 1.

Syntax

`(product` *sequence* `&key` *key start end*`)`

Arguments and Values

sequence A sequence of numbers.

key A function of one argument applied to each element before multiplication. The default is `identity`.

start The starting index (inclusive). The default is 0.

end The ending index (exclusive), or `nil` for the end of the sequence.

Examples

```
(product '(1 2 3 4 5)) ; Returns 120
(product '(1 2 3 4 5) :start 2) ; Returns 60
```

4.2.11 The `sawtooth-wave` Function

The `sawtooth-wave` function is an alias for `fractional-value`.

4.2.12 The `sum` Function

The `sum` function returns the sum of the elements of a sequence, optionally transformed by a key function and restricted to a subsequence. An empty range yields 0.

Syntax

`(sum` *sequence* `&key` *key start end*`)`

Arguments and Values

sequence A sequence of numbers.

key A function of one argument applied to each element before addition. The default is `identity`.

start The starting index (inclusive). The default is 0.

end The ending index (exclusive), or `nil` for the end of the sequence.

Examples

```
(sum (loop for i from 1 to 100 collect i)) ; Returns 5050
(sum '(1 2 3 4 5) :end 3)                  ; Returns 6
(sum '(1 2 3) :key (lambda (i) (* i 2)))   ; Returns 12
```

4.2.13 The `unsigned-integer?` Function

The `unsigned-integer?` function returns true if its argument is a nonnegative integer. It is an alias for `nonnegative-integer?`.

Syntax

```
(unsigned-integer?  x)
```

Examples

```
(unsigned-integer? 12)   ; Returns T
(unsigned-integer? 0)    ; Returns T
(unsigned-integer? 12.7) ; Returns NIL
(unsigned-integer? -12)  ; Returns NIL
```

4.3 Types

4.3.1 The `nonnegative-float` Type

The `nonnegative-float` type comprises floating-point numbers greater than or equal to 0.0.

4.3.2 The `nonnegative-integer` Type

The `nonnegative-integer` type comprises integers greater than or equal to 0.

4.3.3 The `positive-float` Type

The `positive-float` type comprises floating-point numbers strictly greater than 0.0.

4.3.4 The `positive-integer` Type

The `positive-integer` type comprises integers strictly greater than 0.

Chapter 5

The sigma/os Package

The sigma/os package provides a few helpers for talking to the operating system: reading files and running short scripts in other languages. The script runners use portable uiop:run-program (from ASDF's UIOP) and are convenience wrappers, not full foreign interfaces.

5.1 Functions

5.1.1 The perl Function

The perl function runs a short Perl snippet by invoking the interpreter at *perl-path* with -e. The arguments are concatenated into a single string of source before they are passed to Perl. Standard output and error are inherited from Lisp. Implementation is via uiop:run-program.

Syntax

(perl &rest *code*)

Examples

```
(perl "print 2 + 2, qq(\\n);")
```

5.1.2 The python Function

The python function runs a short Python snippet by invoking the interpreter at *python-path* with -c. The arguments are concatenated into a single string of source before they are passed to Python. Standard output and error are inherited from Lisp. Implementation is via uiop:run-program.

Syntax

(python &rest *code*)

Examples

```
(python "print(2␣+␣2)")
```

5.1.3 The `read-file` Function

The `read-file` function reads the entire contents of a file and returns them as a single string.

Syntax

(read-file *filename*)

Arguments and Values

filename A pathname designator for an existing file.

Returns

A string containing the full contents of the file.

5.1.4 The `read-lines` Function

The `read-lines` function reads the entire contents of a file and returns a list of its lines, without the terminating newlines.

Syntax

(read-lines *filename*)

Arguments and Values

filename A pathname designator for an existing file.

Returns

A list of strings, one per line.

5.1.5 The `ruby` Function

The `ruby` function runs a short Ruby snippet by invoking the interpreter at `*ruby-path*` with `-e`. The arguments are concatenated into a single string of source before they are passed to Ruby. Standard output and error are inherited from Lisp. Implementation is via `uiop:run-program`.

Syntax

`(ruby &rest` *code*`)`

Examples

```
(ruby "puts␣2␣+␣2")
```

5.2 Parameters

5.2.1 The `*perl-path*` Parameter

The `*perl-path*` parameter holds the pathname of the Perl interpreter used by `perl`. The default is `/usr/bin/perl`.

5.2.2 The `*python-path*` Parameter

The `*python-path*` parameter holds the pathname of the Python interpreter used by `python`. The default is `/usr/bin/python`.

5.2.3 The `*ruby-path*` Parameter

The `*ruby-path*` parameter holds the pathname of the Ruby interpreter used by `ruby`. The default is `/usr/bin/ruby`.

Chapter 6

The `sigma/probability` Package

The `sigma/probability` package offers a small set of helpers for working with probabilities in the unit interval.

6.1 Macros

6.1.1 The `decaying-probability?` Macro

The `decaying-probability?` macro performs a probabilistic test with `probability?`. When the test succeeds, it multiplies the place `probability` by `factor` (default $1/2$) in place, so that successive calls become less likely, and returns true. When the test fails, it returns `nil` without changing `probability`.

Syntax

`(decaying-probability?` *`probability`* `&optional` *`factor`*`)`

Arguments and Values

`probability` A place holding a value of type `probability`.

`factor` The multiplicative decay applied after a successful trial. The default is $1/2$.

Examples

```
(let ((p 1.0))
  (loop while (decaying-probability? p)
        collect p))
;; Collects a geometrically decaying sequence until a trial fails.
```

6.2 Functions

6.2.1 The `probability?` Function

The `probability?` function returns true with the given probability. It draws a uniform random number in $[0, 1)$ and returns true when that number is less than or equal to `probability`.

Syntax

`(probability?` *probability*`)`

Arguments and Values

probability A value of type `probability`.

Examples

```
(probability? 0.0) ; Always returns NIL
(probability? 1.0) ; Always returns T
(probability? 0.5) ; Returns T or NIL with equal chance
```

6.3 Types

6.3.1 The `probability` Type

The `probability` type comprises values suitable for use as probabilities: floating-point numbers in $[0.0, 1.0]$, the integers 0 and 1, and bits.

Chapter 7

The `sigma/random` Package

The `sigma/random` package provides helpers for random sampling, shuffling, and drawing from simple distributions.

7.1 Macros

7.1.1 The `nshuffle` Macro

The `nshuffle` macro randomly shuffles its argument in place. It expands to an assignment of `(shuffle ...)` back into the place.

Syntax

`(nshuffle` *argument*`)`

Examples

```
(let ((xs '(1 2 3 4 5)))
  (nshuffle xs)
  xs) ; Returns the same elements in a random order
```

7.2 Functions

7.2.1 The `gauss` Function

The `gauss` function draws a sample from a Gaussian (normal) distribution with mean `mu` and standard deviation `sigma`. The implementation

uses the Box–Muller method, caching the second sample of each pair between calls.

Syntax

(gauss *mu sigma*)

Arguments and Values

mu The mean of the distribution.

sigma The standard deviation of the distribution.

7.2.2 The `random-argument` Function

The `random-argument` function returns one of its arguments chosen uniformly at random, or `nil` if no arguments are given.

Syntax

(random-argument &rest *rest*)

Examples

```
(random-argument 'rock 'paper 'scissors)
```

7.2.3 The `coin-toss` Function

The `coin-toss` function returns `t` or `nil` with equal probability, like a fair coin toss.

Syntax

(coin-toss)

7.2.4 The `random-in-range` Function

The `random-in-range` function returns a random number in the half-open interval [*lower*, *upper*). Either bound may be a sequence, in which case the most extreme member of that sequence is used (`maximum` for the lower bound, `minimum` for the upper). If the lower bound is greater than the upper, the bounds are swapped. If they are equal, that common value is returned.

Syntax

```
(random-in-range lower upper)
```

Examples

```
(random-in-range 0 10)     ; A number in [0, 10)
(random-in-range 0.0 1.0)  ; A float in [0.0, 1.0)
```

7.2.5 The `random-in-ranges` Function

The `random-in-ranges` function, given one or more restricting ranges each as a two-element list, returns a random number in a range common to all of them.

Syntax

```
(random-in-ranges &rest ranges)
```

Examples

```
(random-in-ranges '(0 10) '(5 15)) ; A number in a common subrange
```

7.2.6 The `random-range` Function

The `random-range` function returns a two-element list (*lo hi*) of random bounds within [*lower*, *upper*). Without `:containing`, the two bounds are independent samples ordered so that $lo \leq hi$. With `:containing` (a number or sequence), the result is chosen so that the returned range covers that value or those values.

Syntax

```
(random-range lower upper &key containing)
```

7.2.7 The `randomize-array` Function

The `randomize-array` function fills an array in place with values from `(random argument-for-random)` and returns the array.

Syntax

```
(randomize-array array argument-for-random)
```

7.2.8 The `random-array` Function

The `random-array` function returns a new array of the given dimensions whose contents are randomized as by `randomize-array`.

Syntax

`(random-array` *dimensions argument-for-random*`)`

Examples

```
(random-array '(3 3) 10) ; A 3-by-3 array of integers in [0, 10)
```

7.3 Generics

7.3.1 The `random-element` Generic

The `random-element` generic returns a randomly chosen element from a sequence, or `nil` if the sequence is empty. Methods are provided for lists and arrays.

Syntax

`(random-element` *sequence*`)`

Examples

```
(random-element '(a b c d))
(random-element #(1 2 3 4 5))
```

7.3.2 The `shuffle` Generic

The `shuffle` generic returns a new container with the same elements as its argument in random order. The original is not modified; use `nshuffle` to shuffle a place in place. Methods are provided for lists and arrays.

Syntax

`(shuffle` *container*`)`

Examples

```
(shuffle '(1 2 3 4 5))
(shuffle #(a b c d e))
```

Chapter 8

The `sigma/sequence` Package

The `sigma/sequence` package provides utilities for lists, vectors, and other sequences: extrema, sorting by a parallel key, slicing, splitting, and a few destructive helpers.

8.1 Macros

8.1.1 The `nconcf` Macro

The `nconcf` macro destructively concatenates `list-2` onto `list-1` and stores the result back into the place `list-1`, analogous to `nconc` with assignment.

Syntax

`(nconcf` *`list-1 list-2`*`)`

Examples

```
(let ((a '(1 2 3))
      (b '(4 5 6)))
  (nconcf a b)
  a) ; Returns '(1 2 3 4 5 6)
```

8.1.2 The `set-nthcdr` Macro

The `set-nthcdr` macro sets the nth `cdr` of a list to a new value. When n is 0, it sets the list place itself. On implementations that allow it, this is also installed as the `setf` expander for `nthcdr`.

Syntax

(set-nthcdr *n list new-value*)

8.2 Functions

8.2.1 The arefable? Function

The arefable? function returns true if position is a valid multidimensional index list for array via aref: the same rank as the array, with each coordinate in range for its dimension.

Syntax

(arefable? *array position*)

8.2.2 The array-values Function

The array-values function returns a list of the values in an array at the specified positions. Each position is a list of subscripts suitable for aref.

Syntax

(array-values *array positions*)

Examples

```
(array-values #2A((1 2) (3 4))
              '((0 0) (1 1))) ; Returns '(1 4)
```

8.2.3 The nth-from-end Function

The nth-from-end function is similar to nth, but counts from the end of the list. Index 0 is the last element.

Syntax

(nth-from-end *n list*)

Examples

```
(nth-from-end 0 '(0 1 2 3 4 5 6 7 8 9 10)) ; Returns 10
(nth-from-end 3 '(0 1 2 3 4 5 6 7 8 9 10)) ; Returns 7
```

8.2.4 The `nthable?` Function

The `nthable?` function returns true if n is a valid index for `list` via `nth`: a nonnegative integer strictly less than the length of the list.

Syntax

`(nthable?` *n list*`)`

8.2.5 The `sequence?` Function

The `sequence?` function returns true if its argument is of type `sequence` (a list or vector).

Syntax

`(sequence?` *sequence*`)`

8.2.6 The `empty-sequence?` Function

The `empty-sequence?` function returns true if its argument is a sequence that contains no elements. For arrays, any dimension of size zero counts as empty.

Syntax

`(empty-sequence?` *sequence*`)`

Examples

```
(empty-sequence? '())  ; Returns T
(empty-sequence? #())  ; Returns T
(empty-sequence? '(a)) ; Returns NIL
```

8.2.7 The `join-symbol-to-all-following` Function

This function takes a symbol and a list, and for every occurrence of the symbol in the list, it joins it to the item following it. Symbol pieces use `symbol-name`, so `*print-case*` does not change the result. Non-symbol tokens (such as numbers) are printed with the current printer controls.

Syntax

`(join-symbol-to-all-following` *symbol list*`)`

Examples

```
(join-symbol-to-all-following :# '(:# 10 :# 20 :# 30))
;; Returns '(:#10 :#20 :#30)

(let ((*print-case* :downcase))
  (join-symbol-to-all-following :foo (list :foo 'bar :foo :baz)))
;; Still returns '(:foobar :foobaz)
```

Affected By

For non-symbol tokens only: `*print-escape*`, `*print-radix*`, `*print-base*`, `*print-circle*`, `*print-pretty*`, `*print-level*`, `*print-length*`, `*print-gensym*`, `*print-array*`. Symbol pieces are not affected by `*print-case*`.

8.2.8 The `join-symbol-to-all-preceeding` Function

This function takes a symbol and a list, and for every occurrence of the symbol in the list, it joins it to the item preceding it. Symbol pieces use `symbol-name`, so `*print-case*` does not change the result. Non-symbol tokens (such as numbers) are printed with the current printer controls (for example `*print-base*`).

Syntax

`(join-symbol-to-all-preceeding` *symbol list*`)`

Examples

```
(join-symbol-to-all-preceeding :% '(10 :% 20 :% 30 :%))
;; Returns '(:10% :20% :30%)

(let ((*print-base* 8))
  (join-symbol-to-all-preceeding :% (list 64 :%)))
;; Returns '(:100%)

(let ((*print-case* :downcase))
  (join-symbol-to-all-preceeding :foo (list :bar :foo :baz :foo)))
;; Still returns '(:barfoo :bazfoo)
```

Affected By

For non-symbol tokens only: `*print-escape*`, `*print-radix*`, `*print-base*`, `*print-circle*`, `*print-pretty*`, `*print-level*`, `*print-length*`, `*print-gensym*`, `*print-array*`. Symbol pieces are not affected by `*print-case*`.

8.2.9 The `list-to-vector` Function

The `list-to-vector` function takes a list and returns an equivalent vector.

Syntax

`(list-to-vector` *list*`)`

Examples

```
(list-to-vector '(1 2 3)) ; Returns #(1 2 3)
```

8.2.10 The `max*` Function

The `max*` function is a shortcut for `max`. It takes in one or more lists and finds the maximum value within all of them, so that you need not manually use `apply` and `concatenate`.

Syntax

`(max* &rest` *lists*`)`

Examples

```
(max* '(1 2 3 100 4 5)) ; Returns 100
(max* '(1 2 3 4)
      '(5 6 99 7)
      '(8 9 10)) ; Returns 99
```

8.2.11 The `min*` Function

The `min*` function is a shortcut for `min`. It takes in one or more lists and finds the minimum value within all of them, so that you need not manually use `apply` and `concatenate`.

Syntax

`(min* &rest` *lists*`)`

Examples

```
(min* '(1 2 3 -100 4 5)) ; Returns -100
(min* '(1 2 3 4)
      '(5 6 -99 7)
      '(8 9 10)) ; Returns -99
```

8.2.12 The `set-equal` Function

The `set-equal` function returns true if two lists contain the same elements when viewed as sets, using an optional key and either `test` or `test-not` for element comparison. Order and duplicate multiplicity are ignored.

Syntax

`(set-equal` *list-1 list-2* `&key` *key test test-not*`)`

Examples

```
(set-equal '(1 2 3) '(3 2 1))     ; Returns T
(set-equal '(1 2) '(1 2 3))       ; Returns NIL
(set-equal '("a" "b") '("A" "B")
           :test #'string-equal)  ; Returns T
```

8.2.13 The `simple-vector-to-list` Function

The `simple-vector-to-list` function takes a vector and returns an equivalent list, reading elements with `svref`.

Syntax

`(simple-vector-to-list` *vector*`)`

8.2.14 The `sort-order` Function

The `sort-order` function returns the indices that would sort the sequence. Sorting those indices by the corresponding elements under `predicate` and `key` yields the sorted order of the original sequence.

Syntax

`(sort-order` *sequence predicate* `&key` *key*`)`

Examples

```
(sort-order '(30 10 20) #'<) ; Returns '(1 2 0)
```

8.2.15 The `the-last` Function

The `the-last` function returns the last element of a list (the `car` of its `last` cons), or `nil` if the list is empty.

Syntax

`(the-last` *list*`)`

Examples

```
(the-last '(a b c d)) ; Returns D
(the-last '())        ; Returns NIL
```

8.2.16 The `vector-to-list` Function

The `vector-to-list` function takes a vector and returns an equivalent list.

Syntax

`(vector-to-list` *vector*`)`

Examples

```
(vector-to-list #(1 2 3)) ; Returns '(1 2 3)
```

8.3 Generics

8.3.1 The `best` Generic

The `best` generic returns the "best" element of a sequence according to a predicate and optional key. This is equivalent to taking the first element after sorting the sequence with the same predicate and key, but runs in linear time. Methods are provided for lists and vectors. An empty sequence yields `nil`.

Syntax

`(best` *sequence predicate* `&key` *key*`)`

Examples

```
(best '(3 1 4 1 5) #'<) ; Returns 1
(best '(3 1 4 1 5) #'>) ; Returns 5
```

8.3.2 The `minimum` Generic

The `minimum` generic returns the minimum element of a sequence, optionally after applying a key, considering only the subsequence from `start` to `end`.

Syntax

`(minimum` *sequence* `&key` *key start end*`)`

Examples

```
(minimum '(3 1 4 1 5)) ; Returns 1
```

8.3.3 The `minimum?` Generic

The `minimum?` generic returns true if the element at `position` in the sequence is a minimum under `key` within the subsequence from `start` to `end`. When `position` is omitted, it defaults to the last index.

Syntax

`(minimum?` *sequence* `&key` *position key start end*`)`

8.3.4 The `maximum` Generic

The `maximum` generic returns the maximum element of a sequence, optionally after applying a key, considering only the subsequence from `start` to `end`.

Syntax

`(maximum` *sequence* `&key` *key start end*`)`

Examples

```
(maximum '(3 1 4 1 5)) ; Returns 5
```

8.3.5 The `maximum?` Generic

The `maximum?` generic returns true if the element at `position` in the sequence is a maximum under `key` within the subsequence from `start` to `end`. When `position` is omitted, it defaults to the last index.

Syntax

`(maximum? sequence &key position key start end)`

8.3.6 The `sort-on` Generic

The `sort-on` generic sorts one sequence according to a parallel ordering sequence. Corresponding elements are paired; the pairs are sorted by the predicate on the ordering values (optionally through a key), and the sorted elements of the sequence to sort are returned. Methods accept lists and vectors in either role.

Syntax

`(sort-on sequence-to-sort ordering-sequence predicate &key key)`

Examples

```
(sort-on '(a b c) '(3 1 2) #'<) ; Returns '(b c a)
```

8.3.7 The `slice` Generic

The `slice` generic returns a modular subset of a sequence, taking every `slice`-th element (`slice` defaults to 1). The stride may be any positive rational number. Methods are provided for lists and vectors.

Syntax

`(slice sequence &optional slice)`

Examples

```
(slice '(1 2 3 4 5 6 7 8 9) 2) ; Returns '(1 3 5 7 9)
(slice #(1 2 3 4 5 6 7 8 9) 2) ; Returns #(1 3 5 7 9)
```

8.3.8 The `split` Generic

The `split` generic splits a sequence into subsequences wherever an element is a member of `separators` (under `key` and `test`). When `remove-separators?` is true (the default), separator elements are omitted from the results. Methods are provided for lists (and, in the string package, for strings).

Syntax

`(split` *sequence separators* `&key` *key test remove-separators?*`)`

Examples

```
(split '(1 2 0 3 4 0 5) '(0))
;; Returns '((1 2) (3 4) (5))
```

8.3.9 The `worst` Generic

The `worst` generic returns the "worst" element of a sequence according to a predicate and optional key. This is equivalent to taking the last element after sorting the sequence with the same predicate and key, but runs in linear time. Methods are provided for lists and vectors. An empty sequence yields `nil`.

Syntax

`(worst` *sequence predicate* `&key` *key*`)`

Examples

```
(worst '(3 1 4 1 5) #'<) ; Returns 5
(worst '(3 1 4 1 5) #'>) ; Returns 1
```

Chapter 9

The `sigma/string` Package

The `sigma/string` package contains useful tools for working with strings: ranges of characters, joining and repeating, trimming whitespace, and converting other objects to text.

9.1 Constants

9.1.1 The `+whitespace+` Parameter

The `+whitespace+` constant is a list of characters treated as whitespace by the trimming functions: `#\Space`, `#\Newline`, `#\Backspace`, `#\Tab`, `#\Linefeed`, `#\Page`, `#\Return`, and `#\Rubout`.

9.2 Functions

9.2.1 The `character-range` Function

The `character-range` function returns a list of characters from the *start* to the *end* character. Note that this returns a list, not a string. The endpoints may be given in either order; the result is always ascending.

Syntax

`(character-range` *start end*`)` $\Longrightarrow$ `'(`*start ... end*`)`

Arguments and Values

start The character to start the range with, inclusive.

end The character to end the range with, inclusive.

Examples

```
(character-range #\a #\e) ⟹ '(#\a #\b #\c #\d #\e)
(character-range #\e #\a) ⟹ '(#\a #\b #\c #\d #\e)
```

9.2.2 The `character-ranges` Function

The `character-ranges` function is a convenience wrapper for `character-range`, concatenating several calls and making the resultant list contain only unique instances, sorted in ascending order.

Syntax

(character-ranges $start_1$ end_1 ...) ⟹ '($character_1$ ...)

Arguments and Values

start$_n$ The character to start the nth range with, inclusive.

end$_n$ The character to end the nth range with, inclusive.

Examples

```
(character-ranges #\a #\c #\x #\z) ⟹ '(#\a #\b #\c #\x #\y
#\z)
(character-ranges #\a #\c #\a #\c) ⟹ '(#\a #\b #\c)
```

9.2.3 The `escape-tildes` Function

The `escape-tildes` function returns a copy of a string in which every tilde (`#\~`) is doubled, suitable for use as a `format` control string that should print tildes literally.

Syntax

(escape-tildes *string*)

Examples

```
(escape-tildes "foo␣bar")  ; Returns "foo bar"
(escape-tildes "foo~bar")  ; Returns "foo~~bar"
```

9.2.4 The `replace-char` Function

The `replace-char` function replaces every instance of `from-char` in `string` with `to-char`. The string is modified in place and returned.

Syntax

```
(replace-char string from-char to-char)
```

Examples

```
(replace-char (copy-seq "banana") #\textbackslash a #\textbackslash o)
;; Returns "bonono"
```

9.2.5 The `strcat` Function

The `strcat` function takes any number of arguments, converts each to a string with `to-string`, and concatenates the results. The symbol `string-concatenate` is an alias for this function.

Syntax

```
(strcat &rest rest)
  (string-concatenate &rest rest)
```

Examples

```
(strcat "foo" "bar")           ; Returns "foobar"
(strcat "foo" 123 "bar")       ; Returns "foo123bar"
(strcat)                       ; Returns ""
(strcat 1 2 3 4)               ; Returns "1234"
```

9.2.6 The `string-concatenate` Function

The `string-concatenate` function is an alias for `strcat`.

9.2.7 The `strmult` Function

The `strmult` function concatenates its string arguments (via `strcat`) and repeats that result `count` times, returning one combined string. When `count` is less than 1, it returns the empty string.

Syntax

```
(strmult count &rest strings)
```

Examples

```
(strmult 3 "ab")      ; Returns "abababab"
(strmult 2 "x" "y")   ; Returns "xyxy"
(strmult 0 "nope")    ; Returns ""
```

9.2.8 The `string-join` Function

The `string-join` function joins a list of strings into one string, inserting `connecting-string` between each adjacent pair. A single string may be passed and is returned unchanged aside from the single-element join path. The connecting string defaults to the empty string.

Syntax

`(string-join` *strings* `&optional` *connecting-string*`)`

Examples

```
(string-join '("a" "b" "c") "-") ; Returns "a-b-c"
(string-join '("a" "b" "c"))     ; Returns "abc"
```

9.2.9 The `stringify` Function

The `stringify` function takes an argument of any type and converts it to a string using the `~A` directive to `format` inside `with-standard-io-syntax`, so the result does not depend on the caller's printer controls (for example `*print-case*` or `*print-base*`). Also see `to-string`.

Syntax

`(stringify` *argument*`)`

Examples

```
(stringify 12)   ; Returns "12"
(stringify :foo) ; Returns "FOO"
(let ((*print-case* :downcase) (*print-base* 16))
  (stringify :foo)  ; Still "FOO"
  (stringify 255))  ; Still "255"
```

9.2.10 The `string-trim-whitespace` Function

The `string-trim-whitespace` function removes whitespace from both ends of a string, using the characters in `+whitespace+`.

Syntax

```
(string-trim-whitespace string)
```

Examples

```
(string-trim-whitespace "␣␣␣foo␣␣␣") ; Returns "foo"
```

9.2.11 The `string-left-trim-whitespace` Function

The `string-left-trim-whitespace` function removes whitespace from the left side only of a string.

Syntax

```
(string-left-trim-whitespace string)
```

Examples

```
(string-left-trim-whitespace "␣␣␣foo␣␣␣") ; Returns "foo   "
```

9.2.12 The `string-right-trim-whitespace` Function

The `string-right-trim-whitespace` function removes whitespace from the right side only of a string.

Syntax

```
(string-right-trim-whitespace string)
```

Examples

```
(string-right-trim-whitespace "␣␣␣foo␣␣␣") ; Returns "   foo"
```

9.2.13 The `to-string` Function

The `to-string` function converts common types of things into a string. It handles some special cases more usefully than `stringify` for most user-facing output: `nil` becomes the empty string, symbols become their downcased `symbol-name`s, and strings are returned unchanged. Other values are printed with `~A` under `with-standard-io-syntax`.

Syntax

(to-string *s*)

Examples

```
(to-string nil)    ; Returns ""
(to-string :foo)   ; Returns "foo"
(to-string "hello"); Returns "hello"
(to-string 255)    ; Returns "255" even if *print-base* is 16
```

9.3 Methods

9.3.1 The `split` Methods

The `split` method on strings splits a string into a list of substrings on `separators`. Separators may be a single separator or a sequence of them. The `key` and `test` arguments control membership tests, and `remove-separators?` controls whether separator text is kept. The implementation coerces the string to a list, delegates to the list method of `split` from `sigma/sequence`, and coerces each piece back to a string.

Syntax

(split *string separators* &key *key test remove-separators?*)

Examples

```
(split "a,b,c" #\,) ; Returns '("a" "b" "c")
```

Chapter 10

The `sigma/time-series` Package

The `sigma/time-series` package supports simple time series and multivariate time series (time multiseries), together with geometric helpers such as raster lines through grids, norms, and distances.

A *time series* is a nonempty list whose elements share a common type. A *time multiseries* is a nonempty list of arrays of equal dimensions, where each array holds the data for one time step.

10.1 Macros

10.1.1 The `snap-index` Macro

The `snap-index` macro wraps an index into the half-open interval $[0, \mathit{bound})$ by adding or subtracting `bound` as needed. It modifies the place `index` in place.

Syntax

`(snap-index` *`index bound`*`)`

10.2 Functions

10.2.1 The `array-raster-line` Function

The `array-raster-line` function returns a one-dimensional list of values from an array along the straight-line path from `start-point` to `end-point`. The array may be of any rank. Points are generated by `raster-line` and read with `array-values`.

Syntax

(array-raster-line *array start-point end-point* &key *coordinate-assertion from-start from-end*)

10.2.2 The `distance` Function

The `distance` function calculates the distance between two points as the norm of their coordinate-wise difference. The optional power is as for `norm`.

Syntax

(distance *initial-point final-point* &optional *power*)

Examples

```
(distance '(0 0) '(3 4)) ; Returns 5.0
```

10.2.3 The `norm` Function

The `norm` function returns the mathematical vector norm of a sequence. For the infinity norm, pass `:infinity` as the power.

Syntax

(norm *sequence* &optional *power*)

Arguments and Values

sequence A sequence of numbers.

power A positive number, or `:infinity`. The default is 2 (the Euclidean norm).

Examples

```
(norm '(3 4))              ; Returns 5.0
(norm '(3 4) :infinity)    ; Returns 4
```

10.2.4 The `raster-line` Function

The `raster-line` function returns the discrete points on a straight line from `start-point` to `end-point` in an n-dimensional integer grid. It is derived from the three-dimensional scan-conversion algorithm of Kaufman and Shimony, generalized to any nonnegative dimension. Optional `from-start` and `from-end` trim points from either end of the basic path.

Syntax

(raster-line *start-point end-point* &key *coordinate-assertion from-start from-end*)

Arguments and Values

start-point A list of coordinates.

end-point A list of coordinates of the same length.

coordinate-assertion A predicate applied to each coordinate (default `integerp`).

from-start Nonnegative integer offset from the start.

from-end Integer offset from the end.

10.2.5 The `similar-points?` Function

The `similar-points?` predicate returns true if two points are lists of equal length whose coordinates all satisfy an optional assertion (default `numberp`).

Syntax

(similar-points? *p q* &optional *coordinate-assertion*)

10.2.6 The `time-series?` Function

The `time-series?` predicate returns true if its argument could be a time series: a nonempty list whose every element is of the given element type (default `t`).

Syntax

(time-series? *time-series* &optional *element-type*)

Examples

```
(time-series? '(1.0 2.0 3.0) 'float) ; Returns T
(time-series? '())                   ; Returns NIL
```

10.2.7 The `time-multiseries?` Function

The `time-multiseries?` predicate returns true if its argument is a time multiseries: a nonempty list of arrays of equal dimensions.

Syntax

(time-multiseries? *time-multiseries*)

10.2.8 The `tmsref` Function

The `tmsref` function works like `aref`, but for a time series or multiseries. For a one-dimensional time series (a list), it returns the element at the given time. For a multiseries, it returns the array element at `position` within the array for that time step.

Syntax

(tmsref *time-multiseries time* &rest *position*)

Examples

```
(tmsref '(10 20 30) 1) ; Returns 20
```

10.2.9 The `tms-dimensions` Function

The `tms-dimensions` function works like `array-dimensions`, but for a time series or multiseries. The first dimension listed is the time dimension.

Syntax

(tms-dimensions *time-multiseries*)

10.2.10 The `tms-values` Function

The `tms-values` function returns a list of the values in a time series or multiseries at the specified positions. Each position is a list whose first element is the time index; further elements are spatial subscripts for a multiseries. A bare integer is treated as a one-element position list.

Syntax

(tms-values *time-multiseries positions*)

10.3 Types

10.3.1 The `time-multiseries` Type

The `time-multiseries` type comprises objects that satisfy `time-multiseries?`.

Chapter 11

The `sigma/truth` Package

The `sigma/truth` package provides a few small tools for converting and negating truth values.

11.1 Functions

11.1.1 The `[?]` Function

The `[?]` function is Knuth's truth function. It converts its argument to `1` for true and `0` for false. In other words, `nil` maps to `0` and everything else maps to `1`.

Syntax

```
([?]  x)
```

Examples

```
([?] nil) ; Returns 0
([?] t)   ; Returns 1
([?] 12)  ; Returns 1
```

11.1.2 The `toggle` Function

The `toggle` function returns the boolean negation of its argument: `nil` if the argument is true, and `t` if the argument is false.

Syntax

```
(toggle x)
```

Examples

```
(toggle nil) ; Returns T
(toggle t)   ; Returns NIL
(toggle 12)  ; Returns NIL
```

11.2 Generics

11.2.1 The ? Generic

The ? generic converts a generalized boolean into a strict boolean: nil stays nil, and any non-nil value becomes t.

Syntax

(? *x*)

Examples

```
(? nil) ; Returns NIL
(? t)   ; Returns T
(? 12)  ; Returns T
(? '()) ; Returns NIL
```

Chapter 12

The `sigma` Package

The `sigma` package is the umbrella package for the library. It uses every subordinate package and re-exports a short list of entry points for loading the whole system into the current package.

12.1 Variables

12.1.1 The `*sigma-packages*` Variable

The `*sigma-packages*` variable holds a list of all Sigma package designators. `use-all-sigma` walks this list. The default value is

```
(:sigma/behave
 :sigma/control
 :sigma/hash
 :sigma/numeric
 :sigma/os
 :sigma/probability
 :sigma/random
 :sigma/sequence
 :sigma/string
 :sigma/time-series
 :sigma/truth
 :sigma)
```

12.2 Functions

12.2.1 The `use-all-sigma` Function

The `use-all-sigma` function calls `use-package` on every package in `*sigma-packages*` from the current package, making all of their external symbols available without package prefixes. This will pollute

common-lisp-user if that is your current package; that is intentional for interactive use.

Syntax

```
(use-all-sigma)
```

Examples

```
(asdf:load-system :sigma)
(sigma:use-all-sigma)
(sum (loop for i from 1 to 100 collect i)) ; Returns 5050
```

Bibliography

[1] GRAHAM, P. *On Lisp*. Prentice-Hall, 1993.

[2] GRAHAM, P. ANSI *Common Lisp*. Prentice-Hall, 1995.